Cape Peninsula University of Technology

CREATING FUTURES

PROJECT REPORT

Crypto Currency Converter

SDN150S | Software Design 1

MODULE
SDN150S – Software Design 1

GROUP
Group 4

LECTURER
Stephen Obono Ekwe

SUBMISSION DATE
8 November 2026

No.	GROUP MEMBER	STUDENT NUMBER
1	**Etienne Roux**	260041637
2	**Garison Pieters**	222535164
3	**Jonathan Strauss**	260083607
4	**Henning Myburgh**	260403237
5	**Joshua Isaacs**	261755692
6	**Robert Kanegamba**	260273600
7	**Sinovuyo Danca**	260554898
8	**Motheo Nkadimeng**	251142884

Declaration

Project: Crypto Currency Converter **Module:** SDN150S – Software Design 1

We, the undersigned, declare that:

1. This report is our own work.
2. All sources of information, including textbooks, datasheets and websites, have been acknowledged in the text and listed in the References.
3. No part of this work has been copied from another student, group or any other person, and we have not allowed anyone to copy ours.
4. The work has not previously been submitted for assessment in any other module or institution.
5. We understand that plagiarism is a form of academic misconduct and that it may result in disciplinary action.

No.	NAME	STUDENT NO.	SIGNATURE	DATE
1	Etienne Roux	260041637		2026-01-11
2	Garison Pieters	222535164		2026-02-11
3	Jonathan Strauss	260083607		2026-05-11
4	Henning Myburgh	260403237		2026-05-11
5	Joshua Isaacs	261755692		2026-05-11
6	Robert Kanegamba	260273600		2026-05-11
7	Sinovuyo Danca	260554898		2026-05-11
8	Motheo Nkadimeng	251142884		2026-05-11

Overview

This report documents the design, implementation and testing of a Java Swing desktop application that converts between ten cryptocurrencies and two fiat currencies (USD and ZAR).

The application was built by a team of eight first-year students using Apache NetBeans.

The report covers the problem and sub-questions, the object-oriented design, the Java concepts used, the tools, and the test results.

Pseudocode, the UML class diagram and the team contribution as well as the source code are recorded in the Appendix.

Table of Content

1. Introduction....6
2. Design....7
   2.1 Converter Class....7
   2.2 CryptoCoinUI Class....8
   2.3 UML Class Diagram....9
3. Implementation....9
   3.1 Keeping Calculation Separate (Q1)....9
   3.2 Converting in Any Direction (Q2)....9
   3.3 Handling Invalid Input (Q3)....10
   3.4 Keeping the Window Simple (Q4)....10
4. Tools and technologies....11
5. Testing....12
6. Conclusion....15
7. References....15
8. Appendix....16

   Appendix A – Team Contribution....16
   Appendix B - Pseudocode....17
   Appendix C - UML class diagram....18
   Appendix D – Source Code....18
      Converter.java....18
      CryptoCoinUI.java....19
   Appendix E – Version Control Evidence....21

Table of Figures

Figure 1: The currency converter window 6
Figure 2: The Converter class (Converter.java) 7
Figure 3: The CryptoCoinUI class (CryptoCoinUI.java) 8
Figure 4: Conversion Demo 10
Figure 5: Test Case 1 normal conversion 12
Figure 6: Test Case 2 Text Input 13
Figure 7: Test Case 3 Negative Value Input 13
Figure 8: Test Case 4 Empty Input 14
Figure 9: Test Case 5 Exit 14
Figure 10: UML Class Diagram 18
Figure 11: Git-Branches 21
Figure 12: Git-Commits 21
Figure 13: Git-Log 22
Figure 14: Github Repo 22

1. Introduction

This project builds a Cryptocurrency converter in Java. The user picks two currencies, types an amount and sees the converted value straight away, together with its value in rand.

The aim is to design, build and test a Java program with a graphical window that converts between ten cryptocurrencies (BTC, ETH, BNB, SOL, XRP, ADA, DOGE, TRX, LINK and LTC) and two ordinary (fiat) currencies, USD and ZAR.

The application was built with Apache NetBeans and the Java Swing toolkit. The result is displayed to eight decimal places, since crypto amounts can be very small, and the same amount is also shown in rand (ZAR) to two decimal places.

The program demonstrates the Java concepts taught in Software Design 1: classes, constructors, access modifiers, control statements and Swing frame forms.

Four sub-questions guide the report:

1. **Q1.** How can the conversion logic be separated from the graphical interface so that each can be developed, tested and changed independently?
2. **Q2.** How can twelve currencies be converted in any direction while keeping the number of stored rates to a minimum?
3. **Q3.** How can the program respond to invalid user input — such as non-numeric or negative amounts — without crashing?
4. **Q4.** How can the interface be designed so that a first-time user can perform a conversion quickly, with minimal steps and clear feedback?

These Question will be answered in more detailed in Section 3: Implementation

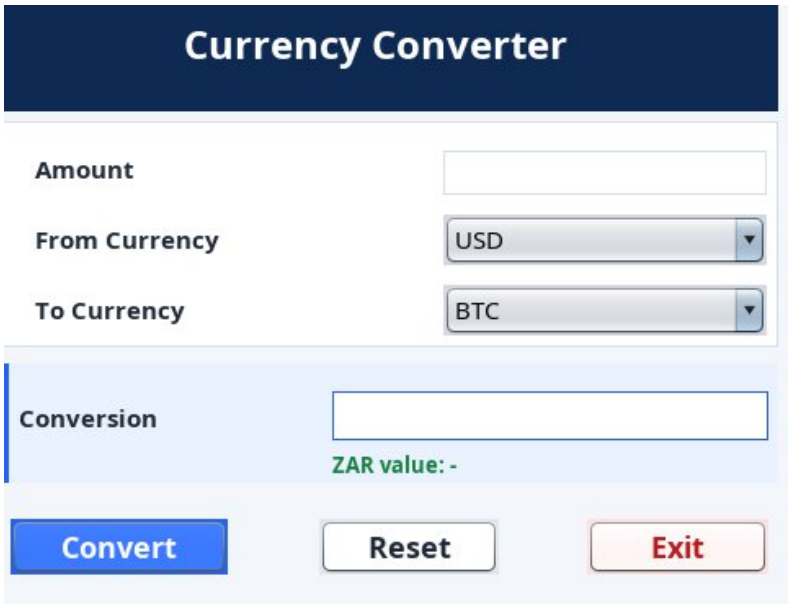


Figure 1: The currency converter window

2. Design

The program is split into two classes. Each class has one clear job, which answers Q1 (keeping the calculation separate from the window).

2.1 Converter Class

Converter does all the mathematics. It has no buttons or windows.

The Converter class, Converter.java file , can be broken down to the following

- *CODES*: A private static final constant array holding the 12 currency codes (10 crypto + 2 fiat).
- getCodes(): A public accessor that lets other classes read the list, since the array itself is private.
- getRate(code): Returns the USD value of one unit of that currency (e.g. 1 BTC = 77962.69 USD). An unknown code returns 1 by default.
- convert(amount, from, to): Converts through USD in two steps: amount × rate of from gives USD, then USD ÷ rate of to gives the result

Because every rate is stored as a value relative to USD, any currency can be converted to any other with only twelve stored rates instead of a rate for every possible pair (12 × 12 = 144).

This directly answers Q2

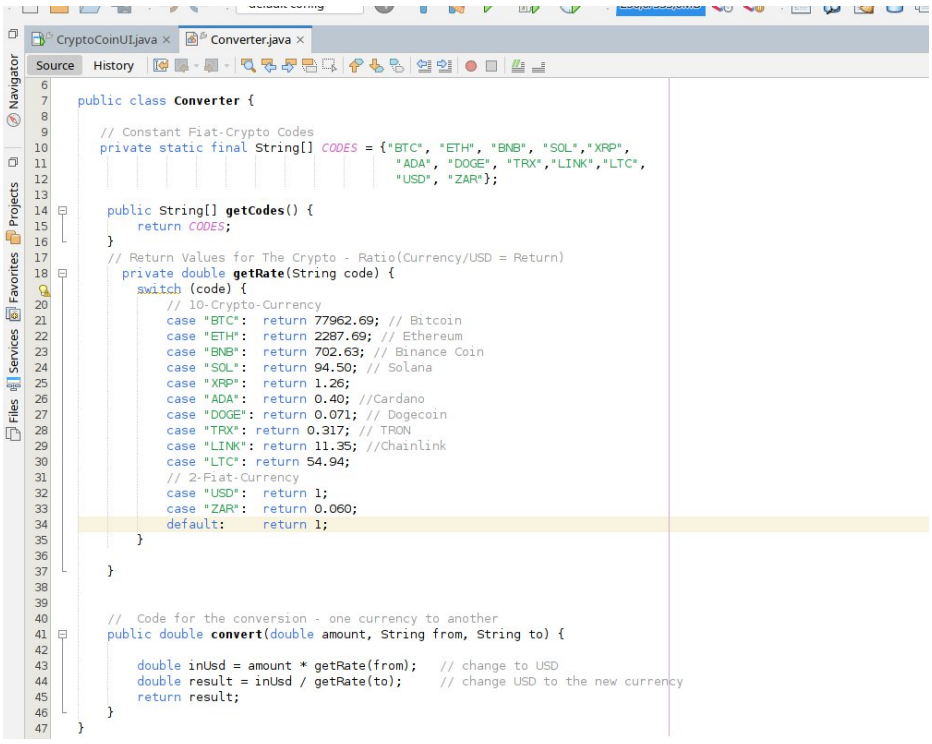


Figure 2: The Converter class (Converter.java)

2.2 CryptoCoinUI Class

CryptoCoinUI extends javax.swing.JFrame and is the window. It handles clicks and asks Converter for the maths.

The CryptoCoinUI class, in the CryptoCoinUI.java file, can be broken down into the following parts:

- Constructor: calls *initComponents()* (the NetBeans-generated layout code), fills the dropdowns with *populateCurrencyComboBoxes()*, and locks the result box with *setEditable(false)*
- **Combo boxes:** the From and To dropdowns are filled from *converter.getCodes()* and start on USD → BTC
- **InsertedValueTF:** pressing Enter in the amount box calls the Convert button's method, so Enter works the same as clicking Convert.
- **Convert button:** reads and trims the amount, validates it, converts it, and displays the result to 8 decimal places plus the ZAR value to 2 decimal places.
- **Reset button**: clears the boxes, resets the ZAR label to "ZAR value: -", sets the dropdowns back to USD → BTC and returns the cursor to the amount box.
- **Exit button:** shows a Yes/No confirmation dialog and only exits if the user clicks Yes.
- *Main*: applies the Nimbus look-and-feel (logging an error if it fails) and creates and shows the window on Swing's UI thread using *EventQueue.invokeLater* with a lambda.

This simple, single-window layout with only four controls and sensible defaults answers Q4.

CryptoCoinUI.java × Converter.java ×

Source Design History

```
        ConvertBtnActionPerformed(evt);
    }

    private void FromValueCBActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
    }

    private void ToValueCBActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
    }

    private void EndValueTFActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
    }

//Convert Button
    private void ConvertBtnActionPerformed(java.awt.event.ActionEvent evt) {
        try {
            double amount = Double.parseDouble(InsertedValueTF.getText().trim());
            if (amount < 0) {
                javax.swing.JOptionPane.showMessageDialog(this, "Amount cannot be negative.");
                return;
            }

            String from = (String) FromValueCB.getSelectedItem();
            String to = (String) ToValueCB.getSelectedItem();

            double result = converter.convert(amount, from, to);
            EndValueTF.setText(String.format("%.8f %s", result, to));

            double zar = converter.convert(amount, from, "ZAR");
            jZarLabel.setText(String.format("ZAR value: R %.2f", zar));

        } catch (NumberFormatException e) {
            javax.swing.JOptionPane.showMessageDialog(this, "Please enter a valid number.");
        }

    }

// Reset Button
    private void ResetBtnActionPerformed(java.awt.event.ActionEvent evt) {
        InsertedValueTF.setText("");
        EndValueTF.setText("");
```

Figure 3: The CryptoCoinUI class (CryptoCoinUI.java)

2.3 UML Class Diagram

The UML class diagram, found in Appendix C, shows the relationship between the two classes: **CryptoCoinUI** holds a *private final Converter converter* field and depends on **Converter** for all calculations, while **Converter** knows nothing about the window. This one-way dependency is what allows either class to change independently.

3. Implementation

3.1 Keeping Calculation Separate (Q1)

Converter contains no Swing imports and no user-interface code. CryptoCoinUI never performs arithmetic itself; it only calls *converter.convert(...)*. This separation means the conversion logic could be reused in a web app or command-line tool without change, and the window could be redesigned without touching the maths.

Evidence: Converter has no Swing imports (Appendix D)

3.2 Converting in Any Direction (Q2)

Every currency is stored with a single USD-based rate. The convert method works in two steps:

```
public double convert(double amount, String from, String to) {

   double inUsd = amount * getRate(from);   // change to USD
   double result = inUsd / getRate(to);     // change USD to the new currency
   return result;
}
```

Example: 1 ETH to BTC = (1 × 2287.69) ÷ 77962.69 = 0.02934339 BTC.

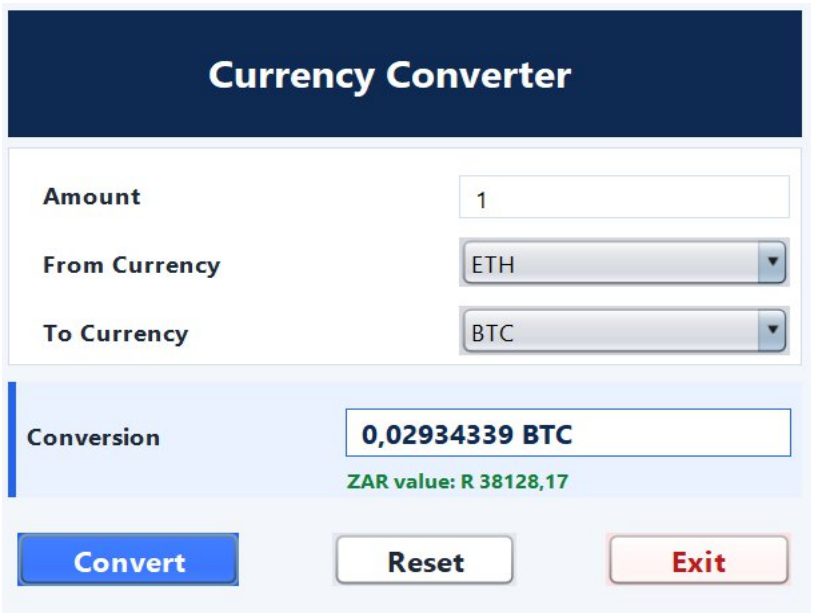


Figure 4: Conversion Demo

3.3 Handling Invalid Input (Q3)

The Convert button wraps its logic in a *try/catch*

- *Double.parseDouble(...)* throws a NumberFormatException if the user types text; the catch block shows "Please enter a valid number." instead of crashing.
- A negative check shows "Amount cannot be negative." and uses return to stop the method.
- The result field is set non-editable so users cannot type into it.

```
// javax.swing. prefix omitted for space
try {
    double amount = Double.parseDouble(InsertedValueTF.getText().trim());
    if (amount < 0) {
        JOptionPane.showMessageDialog(this, "Amount cannot be negative.");
        return;
    }
    // convert and display the result ...
} catch (NumberFormatException e) {
    JOptionPane.showMessageDialog(this, "Please enter a valid number.");
}
```

3.4 Keeping the Window Simple (Q4)

- Defaults are pre-selected (USD → BTC) so the user can convert immediately.
- Pressing Enter in the amount box triggers the conversion, so the mouse is optional.
- The result is read-only, preventing accidental edits.
- Reset returns everything to a known starting state with one click.
- Exit asks for confirmation, preventing accidental closure.

Enter-key support is a single call to the Convert handler:

```
private void InsertedValueTFActionPerformed(java.awt.event.ActionEvent evt) {
    ConvertBtnActionPerformed(evt);
}
```

4. Tools and technologies

1. **Apache NetBeans 31:** Java editor for writing, compiling and running the program; its Form Editor built the window layout.
2. **Java JDK and Swing (Oracle): T**he language, runtime and window components.
3. **Git:** Version control on each computer (commits, branches and merges).
4. **GitHub:** Shared online repository, code review and submission.
5. **PlantUML:** Text-based source for the UML class diagram.
6. **Obsidian.md:** Markdown notes for the pseudocode and drafts.
7. **Microsoft Word:** Writing the report and exporting it to PDF.
8. **WhatsApp:** Used for team communication.
9. **Repository** (Private)**:** **https://github.com/CPUT-Comp-Eng/SDN150S_Group_Java.git** **,** See Appendix E For more information

5. Testing

The program was tested against the following cases:

Nr.	Test	Result
1	Normal conversion	1 BTC → 825.00201058 SOL
2	Text input: “abx”	Message: “Please enter a valid number."
3	Negative input: “-56”	Message: "Amount cannot be negative."
4	Empty input	Message: “Please enter a valid number."
5	Exit	Message: “Exit the Converter?”

Table 1: Expected Results Table

Limitations:

- The currency values are fixed in code.
- Crypto prices are highly volatile, so the rates would need constant updating.
- The rates carry no date; they were captured on 2026-11-02
- *getRate()* returns 1 for an unknown code instead of reporting an error.
- Adding live prices from an API would make this much closer to the converter feature in an app like Luno.

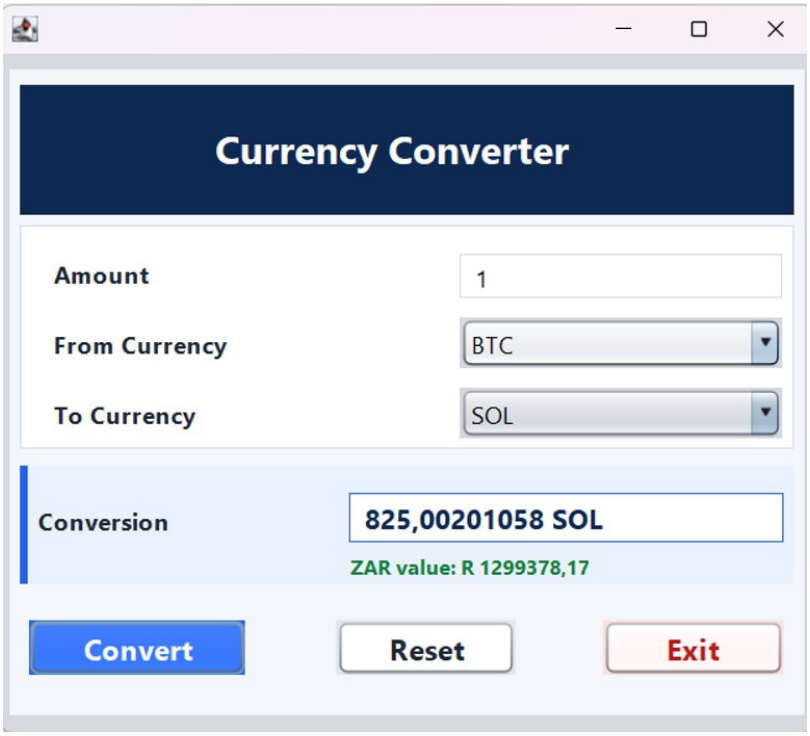


Figure 5: Test Case 1 normal conversion

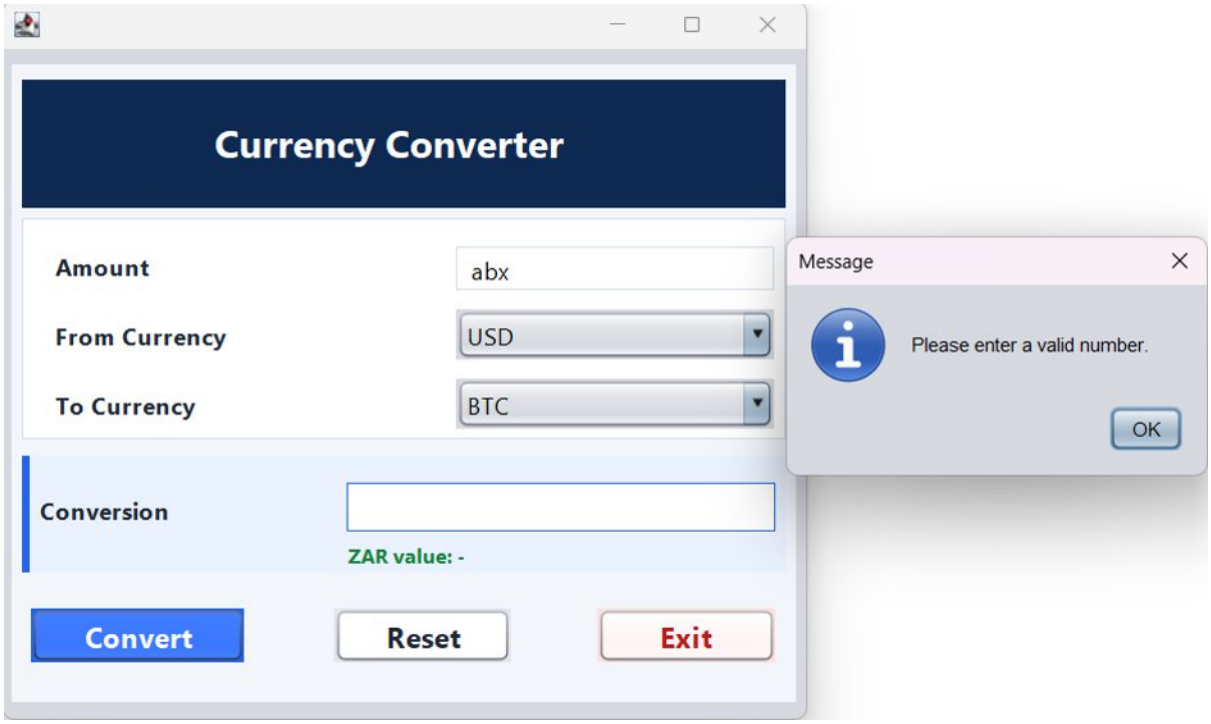


Figure 6: Test Case 2 Text Input

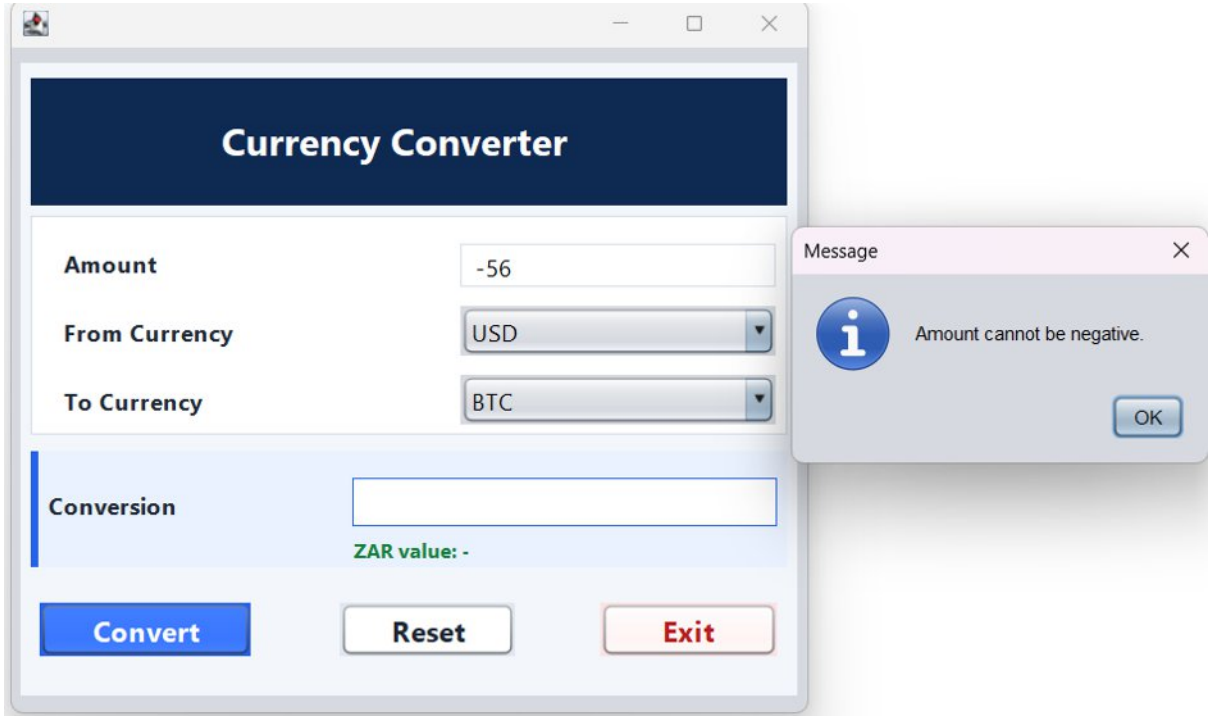


Figure 7: Test Case 3 Negative Value Input

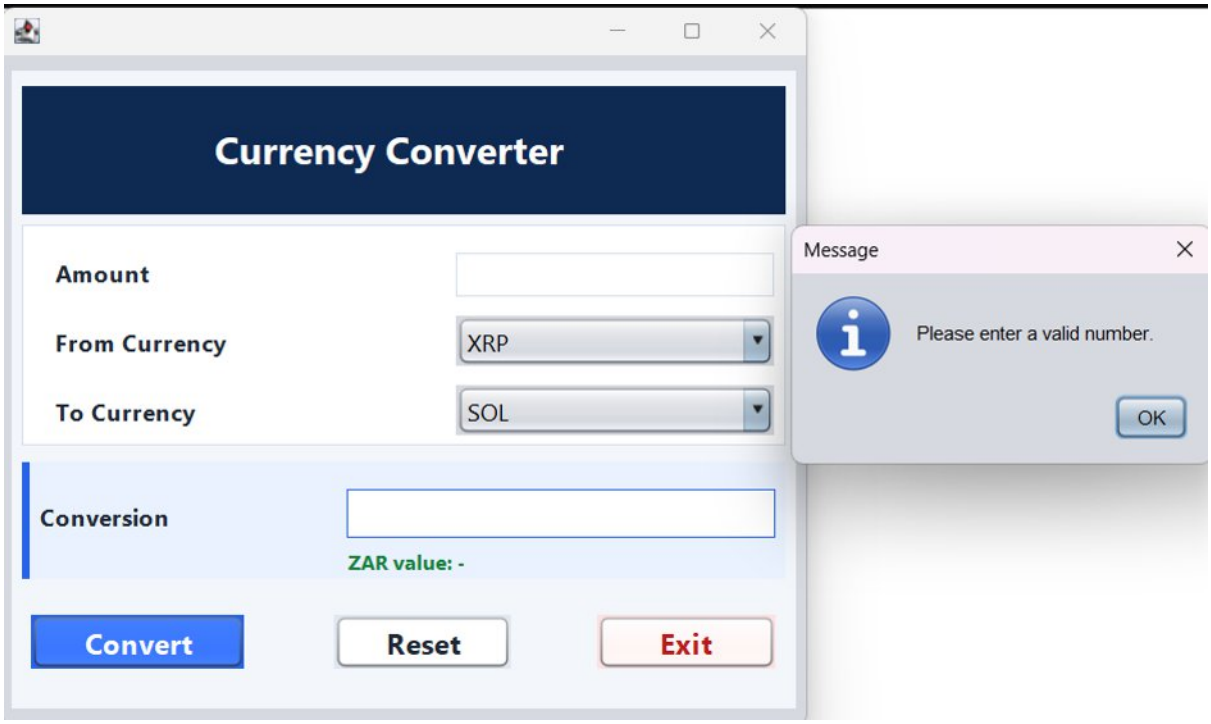


Figure 8: Test Case 4 Empty Input

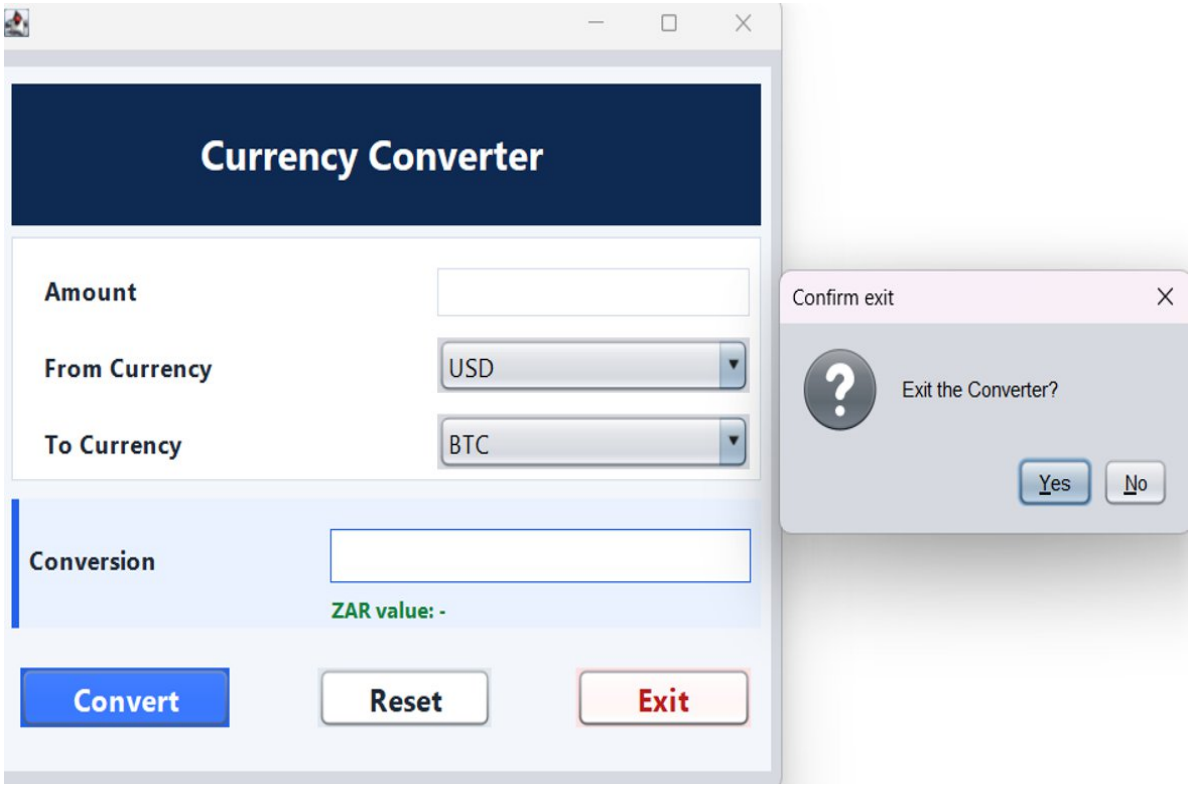


Figure 9: Test Case 5 Exit

6. Conclusion

The project met its aim: a working Java Swing application that converts between ten cryptocurrencies and two fiat currencies in any direction, with a simple, safe user interface.

- **Q1** was answered by separating the Converter (pure calculation) from the CryptoCoinUI (window and event handling), with a one-way dependency between them.
- **Q2** was answered by storing every rate relative to USD and converting through USD in two steps, reducing the stored rates from 144 pairs to 12.
- **Q3** was answered with try/catch for NumberFormatException, a negative-value check, and a read-only result field.
- **Q4** was answered with pre-selected defaults, Enter-key support, a single reset button and an exit confirmation.

The main area for future improvement is replacing the fixed rates with live data from a cryptocurrency API. Overall, the project gave the team practical experience in object-oriented design, Swing event handling and collaborative version control with Git and GitHub.

7. References

1. GeeksforGeeks. 2025. Java Program to Convert Currency using AWT. GeeksforGeeks. https://www.geeksforgeeks.org/java/java-program-to-convert-currency-using-awt/.
2. Project Notes. 2019. Cryptocurrency Converter System. YouTube. https://www.youtube.com/watch?v=nipQiBg9iew.
3. Anon. Java tutorial. W3Schools. https://www.w3schools.com/java/.
4. DJ Oamen. 2024. How to Create a Currency Converter in Java using NetBeans – Full Tutorial. YouTube. https://www.youtube.com/watch?v=fIDX_IGU7K0.
5. Bloch, J. 2018. Effective Java. Addison-Wesley Professional.
6. Payne, B. 2017. Learn Java the Easy Way: A Hands-On Introduction to Programming. No Starch Press.

8. Appendix

Appendix A – Team Contribution

MEMBER	CONTRIBUTION
Etienne Roux (260041637) @RouxTronics	Backend Code (Converter.java), Report, Pseudocode, Research, UML Diagram
Garison Pieters (222535164) @LinkCloudCT	Graphical Interface, Backend Code, Report, Research, Video Demo
Jonathan Strauss (260083607) @JohnnyJoestr	UML Diagram Framework
Henning Myburgh (260403237) @Henning-apx007	Contributed lines of code
Joshua Isaacs (261755692) @joshuai582-stack	Report Template
Robert Kanegamba (260273600) @robitiii	Signed the Declaration
Sinovuyo Danca (260554898) @sinovuyo-asathi-danca	Signed the Declaration
Motheo Nkadimeng (251142884) @motheonkadimeng	Signed the Declaration

Appendix B - Pseudocode

```
Converter

Keep a list of currencies: BTC, ETH, BNB, SOL, XRP, ADA, DOGE,
    TRX, LINK, LTC, USD, ZAR.

Give the list:
    Send back the list.

Find the USD value of a currency:
    BTC is 77962.69
    ETH is 2287.69
    BNB is 702.63
    SOL is 94.50
    XRP is 1.26
    ADA is 0.40
    DOGE is 0.071
    TRX is 0.317
    LINK is 11.35
    LTC is 54.94
    USD is 1
    ZAR is 0.060
    Anything else is 1
    Send back the value.

Convert an amount:
    Multiply the amount by the USD value of the first currency.
    Divide that by the USD value of the second currency.
    Send back the answer.

CryptoCoinUI

Start:
    Create a Converter.
    Fill the dropdowns with the currencies.
    Set From to USD and To to BTC.
    Lock the Conversion box.
    Show the window.

Convert button (also run when Enter is pressed in the amount box):
    Read the amount.
    If it is not a number, show an error and stop.
    If it is negative, show an error and stop.
    Convert it using the chosen currencies.
    Show the result.
    Convert it to ZAR and show that too.

Reset button:
    Clear the boxes and the ZAR label.
    Set From to USD and To to BTC.

Exit button:
    Ask the user to confirm.
    If yes, close the program.
```

Appendix C - UML class diagram

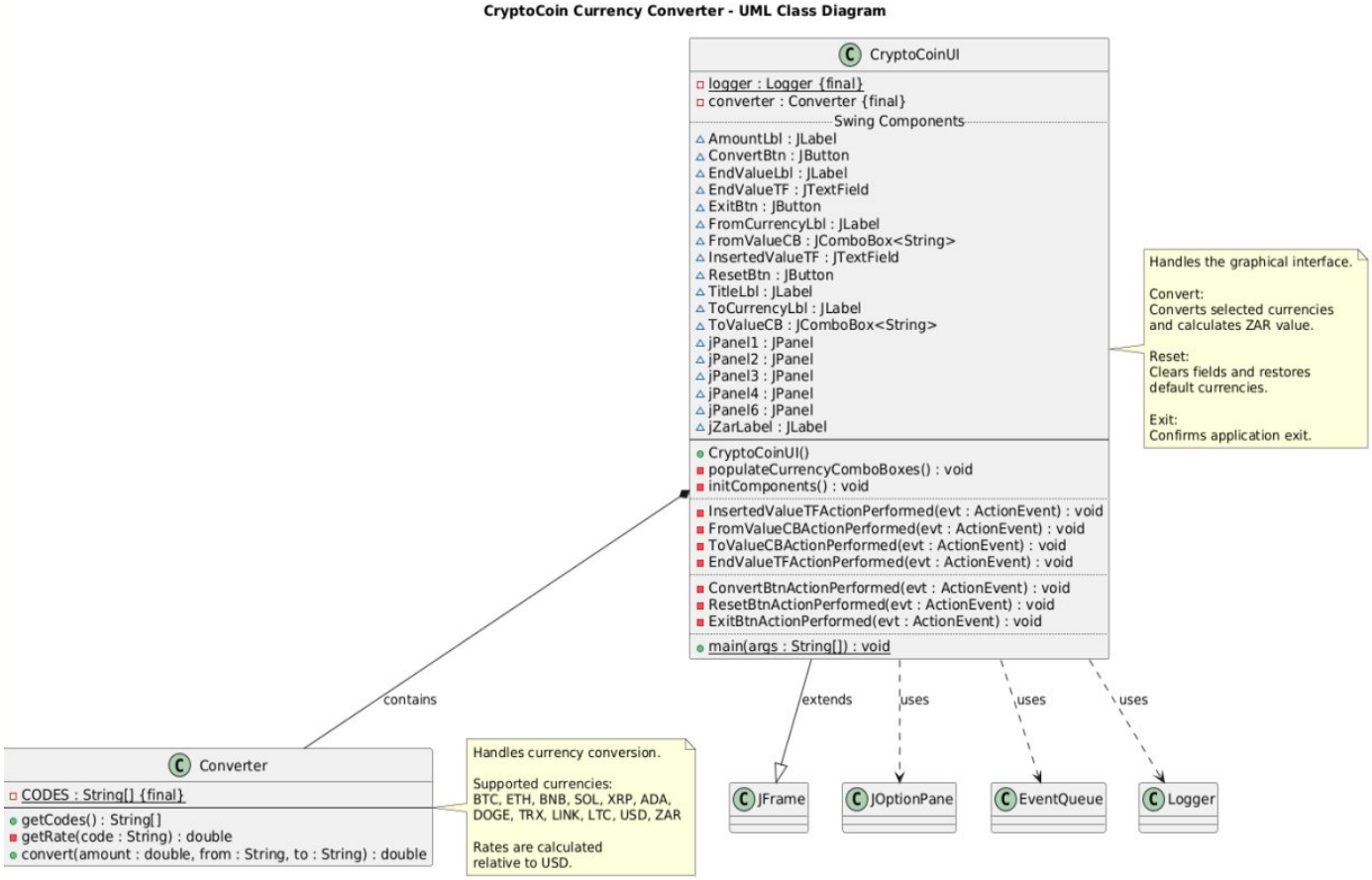


Figure 10: UML Class Diagram

Appendix D – Source Code

Converter.java

```
/*
Description: Currency Converter computation
Authors: Garison Pieters (222535164)
        Etienne G. Roux (260041637)
 */

public class Converter {

  // Constant Fiat-Crypto Codes
  private static final String[] CODES = {"BTC", "ETH", "BNB", "SOL","XRP",
                                         "ADA", "DOGE", "TRX","LINK","LTC",
                                         "USD", "ZAR"};

   public String[] getCodes() {
      return CODES;
   }
   // Return Values for The Crypto - Ratio(Currency/USD = Return)
    private double getRate(String code) {
      switch (code) {
```

```
        // 10-Crypto-Currency
        case "BTC":  return 77962.69; // Bitcoin
        case "ETH":  return 2287.69; // Ethereum
        case "BNB":  return 702.63; // Binance Coin
        case "SOL":  return 94.50; // Solana
        case "XRP":  return 1.26;
        case "ADA":  return 0.40; //Cardano
        case "DOGE": return 0.071; // Dogecoin
        case "TRX": return 0.317; // TRON
        case "LINK": return 11.35; //Chainlink
        case "LTC": return 54.94;
        // 2-Fiat-Currency
        case "USD":  return 1;
        case "ZAR":  return 0.060;
        default:    return 1;
    }

  }


  //  Code for the conversion - one currency to another
  public double convert(double amount, String from, String to) {

    double inUsd = amount * getRate(from);   // change to USD
    double result = inUsd / getRate(to);     // change USD to the new currency
    return result;
  }
```

CryptoCoinUI.java

Constants

```
public class CryptoCoinUI extends javax.swing.JFrame {

  private static final java.util.logging.Logger logger =
java.util.logging.Logger.getLogger(CryptoCoinUI.class.getName());
  private final Converter converter = new Converter();
```

CryptoCoinUI()

```
 public CryptoCoinUI() {
    initComponents();
    populateCurrencyComboBoxes();
    EndValueTF.setEditable(false); // Conversion bar is non-editable
  }
```

Combo Boxes

```
  private void populateCurrencyComboBoxes() {
    FromValueCB.setModel(new javax.swing.DefaultComboBoxModel<>(converter.getCodes()));
    ToValueCB.setModel(new javax.swing.DefaultComboBoxModel<>(converter.getCodes()));
    FromValueCB.setSelectedItem("USD");
    ToValueCB.setSelectedItem("BTC");
  }
```

InsertedValueTF (Inserted Value Text Field)

```
  private void InsertedValueTFActionPerformed(java.awt.event.ActionEvent evt) {
```

```
        ConvertBtnActionPerformed(evt);
    }
```

Convert Button

```
    private void ConvertBtnActionPerformed(java.awt.event.ActionEvent evt) {
        try {
            double amount = Double.parseDouble(InsertedValueTF.getText().trim());
            if (amount < 0) {
                javax.swing.JOptionPane.showMessageDialog(this, "Amount cannot be negative.");
                return;
            }

            String from = (String) FromValueCB.getSelectedItem();
            String to = (String) ToValueCB.getSelectedItem();

            double result = converter.convert(amount, from, to);
            EndValueTF.setText(String.format("%.8f %s", result, to));

            double zar = converter.convert(amount, from, "ZAR");
            jZarLabel.setText(String.format("ZAR value: R %.2f", zar));

        } catch (NumberFormatException e) {
            javax.swing.JOptionPane.showMessageDialog(this, "Please enter a valid number.");
        }

    }
```

Reset Button

```
    private void ResetBtnActionPerformed(java.awt.event.ActionEvent evt) {
        InsertedValueTF.setText("");
        EndValueTF.setText("");
        jZarLabel.setText("ZAR value: -"); // Clears the ZAR Value label
        FromValueCB.setSelectedItem("USD");
        ToValueCB.setSelectedItem("BTC");
        InsertedValueTF.requestFocus();
    }
```

Exit Button

```
    private void ExitBtnActionPerformed(java.awt.event.ActionEvent evt) {
        int choice = javax.swing.JOptionPane.showConfirmDialog(this,
                "Exit the Converter?", "Confirm exit",
                javax.swing.JOptionPane.YES_NO_OPTION);
        if (choice == javax.swing.JOptionPane.YES_OPTION) {
            System.exit(0);
        }
    }
```

Main

```
    public static void main(String args[]) {

        try {
            for (javax.swing.UIManager.LookAndFeelInfo info :
javax.swing.UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {
```

```
                javax.swing.UIManager.setLookAndFeel(info.getClassName());
                break;
            }
        }
    } catch (ReflectiveOperationException | javax.swing.UnsupportedLookAndFeelException ex)
{
        logger.log(java.util.logging.Level.SEVERE, null, ex);
    }

    /* Create and display the form */
    java.awt.EventQueue.invokeLater(() -> new CryptoCoinUI().setVisible(true));
  }
```

Appendix E – Version Control Evidence

The commands below were run in SDN150S_Group_Java and their output is the proof of regular, shared commits.

```
$ git branch -a
$ git shortlog -sne --all
$ git log --pretty=format:"%h | %an | %ad | %s" --date=short
```

```
SDN150S_Group_Java on ÷ main [x?] took 56s
✦5 ❯ git branch -a
  beta
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/Link
  remotes/origin/RouxTronics
  remotes/origin/archives
  remotes/origin/main
```

Figure 11: Git-Branches

```
SDN150S_Group_Java on ÷ main [x?]
✦5 ❯ git shortlog -sne --all
    45  RouxTronics <rouxtronics@gmail.com>
     6  LinkCloudCT <141408005+LinkCloudCT@users.noreply.github.com>
     2  LinkCloud <141408005+LinkCloudCT@users.noreply.github.com>
```

Figure 12: Git-Commits

```
8efa6e2 | RouxTronics | 2026-11-06 | README.md Update
3d4b606 | RouxTronics | 2026-11-06 | Video Demo added, Documentation and Source Code Update
d09ac1d | RouxTronics | 2026-11-02 |  Converter.java
49d242e | RouxTronics | 2026-11-02 | Add Crypto to Converter.java
18b93e2 | RouxTronics | 2026-11-02 | Report update
0ebb281 | RouxTronics | 2026-11-01 | UML Diagram.md update
5d5aa80 | RouxTronics | 2026-11-01 | UML Diagram.md update
93308fc | RouxTronics | 2026-11-01 | 01-Doc/README update
5477548 | RouxTronics | 2026-11-01 | UML Diagram
1007ddc | RouxTronics | 2026-11-01 | Added Tools.md and Update README of Documentation
523f81a | RouxTronics | 2026-11-01 | modded Demo and Code readme
607ac58 | RouxTronics | 2026-11-01 | docs: add README for each top-level directory
d7e2c38 | RouxTronics | 2026-11-01 | refactor: clean submission layout
a079a50 | LinkCloud | 2026-11-01 | Merge pull request #10 from CPUT-Comp-Eng/RouxTronics
a4312ff | RouxTronics | 2026-11-01 | Modified Source code.md and CryptoCoinUI.java
ddecfc5 | RouxTronics | 2026-11-01 | Add Psuedocode,Source documentation
0419d8c | RouxTronics | 2026-11-01 | Change Code in Converter Class
6267336 | LinkCloudCT | 2026-10-31 | Improved the look of the GUI
d441283 | LinkCloud | 2026-10-31 | Merge pull request #9 from CPUT-Comp-Eng/RouxTronics, Editing updated code from Rouxtro
nics
874ae52 | RouxTronics | 2026-10-31 | Updated the Repo
ca9aea2 | RouxTronics | 2026-10-31 | Merge RouxTronics & Link Code
6eb89c1 | RouxTronics | 2026-10-31 | Merge pull request #8 from CPUT-Comp-Eng/Link
0266a2f | RouxTronics | 2026-10-31 | Merge branch 'RouxTronics' into Link
b2d632c | LinkCloudCT | 2026-10-30 | Added BetaCrypto
7c8dfe1 | RouxTronics | 2026-10-29 | Update contributor information in README.md
faa0afe | RouxTronics | 2026-10-29 | update
26e665d | RouxTronics | 2026-10-29 | Add an ZAR Label and also re-edit source in Convert Button
597859e | RouxTronics | 2026-10-29 | Modified Convert button: commas of the conversion output
a8ab9a0 | RouxTronics | 2026-10-29 | add comments
4cd9ed4 | RouxTronics | 2026-10-29 | Created Converter Class
e2f8572 | RouxTronics | 2026-10-29 | Inital Working App
:
```

```
e2f8572 | RouxTronics | 2026-10-29 | Inital Working App
4f2c343 | RouxTronics | 2026-10-28 | Converter current state -  exit, reset, convert buttons with 10-asset rate table
1f7f7ce | RouxTronics | 2026-10-28 | Coded Selector Values
5d9e8c9 | RouxTronics | 2026-10-28 | Coded the reset button
9e8eb1b | RouxTronics | 2026-10-28 | Removed some comments
3a65756 | RouxTronics | 2026-10-28 | Added Exit Button Code
430a25a | RouxTronics | 2026-10-28 | Design-26-10-28-update
07bd061 | RouxTronics | 2026-10-28 | Inital Design-26-10-28
e0a4107 | LinkCloudCT | 2026-10-27 | first draft
45fb295 | RouxTronics | 2026-10-27 | Created Jform for betaCrypto
c6960f1 | RouxTronics | 2026-10-27 | update-gitignore
1eec895 | RouxTronics | 2026-10-27 | README.md
b584f26 | RouxTronics | 2026-10-23 | update: 2026-10-23 20:46:06
28ba6bc | RouxTronics | 2026-10-23 | update 2026-10-23
55827be | RouxTronics | 2026-10-23 | add .gitignore
db74e88 | LinkCloudCT | 2026-10-22 | Stop tracking .obsidian folder and update gitignore
3533c95 | RouxTronics | 2026-10-21 | Update 2026-10-21
8f4fda2 | RouxTronics | 2026-10-21 | Update 2026-10-21
6633ace | RouxTronics | 2026-09-15 | Batman-2026-09-15
(END)
```

Figure 13: Git-Log

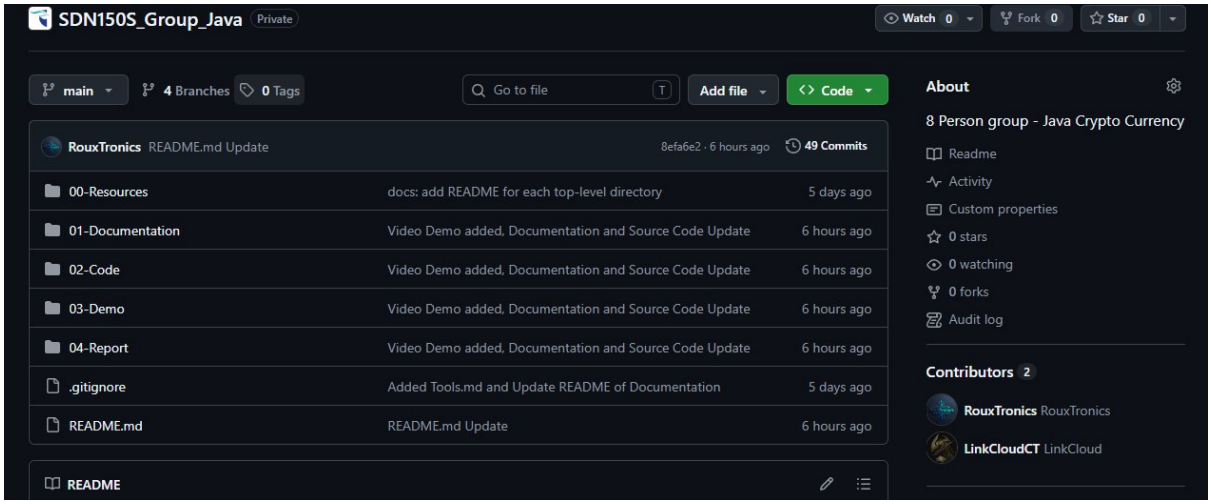


Figure 14: Github Repo